

**LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 2**



ANDROID LAYOUT WITH COMPOSE

Oleh:

Muhammad Bukhari Fitri NIM. 2310817210015

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL 2024**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN I
MODUL 2

Laporan Praktikum Pemrograman Mobile Modul 2: Android Layout With Compose ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Muhammad Bukhari Fitri
NIM : 2310817210015

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Raka Azwar
NIM. 2210817210012

Andreyan Rizky Baskara, S.Kom., M.Kom.
NIP. 19930703 201903 01 011

DAFTAR ISI

LEMBAR PENGESAHAN.....	2
DAFTAR ISI	3
DAFTAR GAMBAR.....	4
DAFTAR TABEL	5
SOAL 1.....	6
A. Source Code.....	8
B. Output Program	18
C. Pembahasan	23
D. Tautan Git	32
SOAL 2.....	33
A. Pembahasan	33

DAFTAR GAMBAR

Gambar 1. Tampilan Awal Aplikasi.....	6
Gambar 2. Tampilan Pilihan Persentase Tip	7
Gambar 3. Tampilan Aplikasi Setelah Dijalankan	8
Gambar 4. Screenshot Hasil Jawaban Soal 1 Tampilan Awal Aplikasi XML	18
Gambar 5. Screenshot Hasil Jawaban Soal 1 Tampilan Pilihan Persentase Tip XML	19
Gambar 6. Screenshot Hasil Jawaban Soal 1 Tampilan Aplikasi Setelah Dijalankan XML	20
Gambar 7. Screenshot Hasil Jawaban Soal 1 Tampilan Awal Aplikasi Compose.....	21
Gambar 8. Screenshot Hasil Jawaban Soal 1 Tampilan Pilihan Persentase Tip Compose ..	22
Gambar 9. Screenshot Hasil Jawaban Soal 1 Tampilan Aplikasi Setelah Dijalankan Compose	23

DAFTAR TABEL

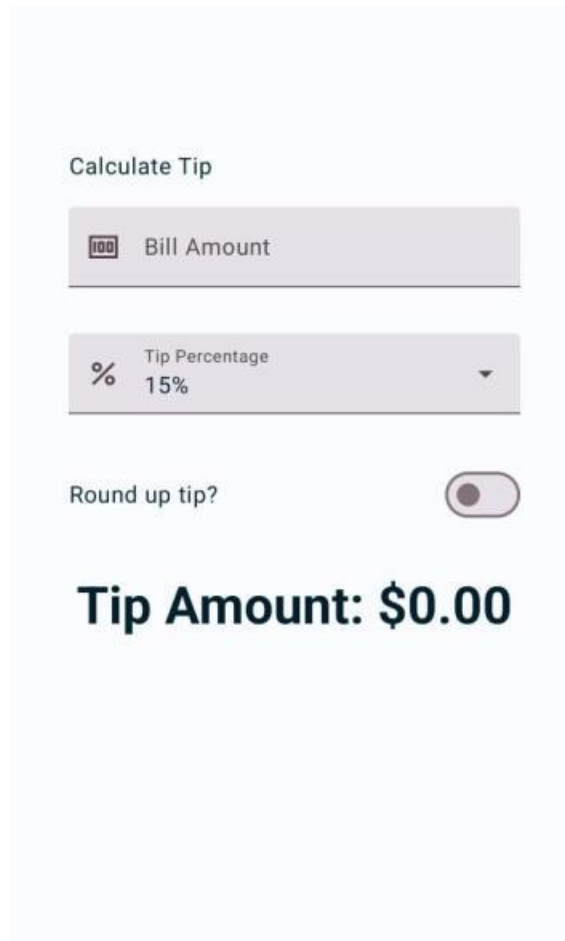
Tabel 1. Source Code Jawaban Soal 1 MainActivity.kt XML	10
Tabel 2. Source Code Jawaban Soal 1 activity_main XML	12
Tabel 4. Source Code Jawaban Soal 1 MainActivity.kt Compose	17

SOAL 1

Soal Praktikum:

Buatlah sebuah aplikasi kalkulator tip menggunakan XML dan Jetpack Compose yang dirancang untuk membantu pengguna menghitung tip yang sesuai berdasarkan total biaya layanan yang mereka terima. Fitur-fitur yang diharapkan dalam aplikasi ini mencakup:

- Input biaya layanan: Pengguna dapat memasukkan total biaya layanan yang diterima dalam bentuk nominal.
- Pilihan persentase tip: Pengguna dapat memilih persentase tip yang diinginkan.
- Pengaturan pembulatan tip: Pengguna dapat memilih untuk membulatkan tip ke angka yang lebih tinggi.
- Tampilan hasil: Aplikasi akan menampilkan jumlah tip yang harus dibayar secara langsung setelah pengguna memberikan input.



Gambar 1. Tampilan Awal Aplikasi

Calculate Tip

	Bill Amount
	60

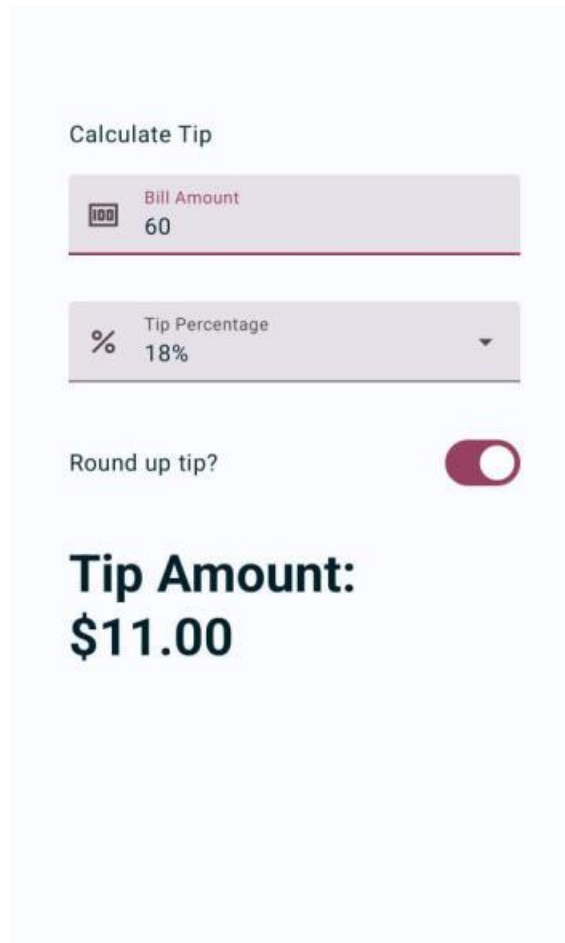
%	Tip Percentage	▲
	15%	

15%

18%

20%

Gambar 2. Tampilan Pilihan Persentase Tip



Gambar 3. Tampilan Aplikasi Setelah Dijalankan

A. Source Code

XML:

MainActivity.kt

```
1 package com.example.calculatortip
2
3 import android.os.Bundle
4 import android.text.Editable
5 import android.text.TextWatcher
6 import android.view.View
7 import androidx.appcompat.app.AppCompatActivity
8 import
9 com.example.calculatortip.databinding.ActivityMainBinding
10 import java.text.NumberFormat
11 import kotlin.math.ceil
12
13 class MainActivity : AppCompatActivity() {
14     private lateinit var binding: ActivityMainBinding
```



```

15     private var amount: String = ""
16     private var tipOption: String = "15%"
17     private var roundUp: Boolean = false
18
19     override fun onCreate(savedInstanceState: Bundle?) {
20         super.onCreate(savedInstanceState)
21         binding = ActivityMainBinding.inflate(layoutInflater)
22         setContentView(binding.root)
23
24         val tipOptions = listOf("15%", "18%", "20%")
25         val adapter = android.widget.AdapterView(this,
26 android.R.layout.simple_dropdown_item_1line, tipOptions)
27         binding.autoCompleteTextView.setAdapter(adapter)
28         binding.autoCompleteTextView.setText(tipOption,
29 false)
30
31         binding.textInputEditText.addTextChangedListener(object
32 : TextWatcher {
33             override fun afterTextChanged(s: Editable?) {
34                 amount = s.toString()
35                 updateTip()
36             }
37             override fun beforeTextChanged(s: CharSequence?,
38 start: Int, count: Int, after: Int) {}
39             override fun onTextChanged(s: CharSequence?,
40 start: Int, before: Int, count: Int) {}
41         })
42
43         binding.autoCompleteTextView.setOnItemClickListener {
44 parent, _, position, _ ->
45             tipOption = parent.getItemAtPosition(position) as
46 String
47             updateTip()
48         }
49
50         binding.switch1.setOnCheckedChangeListener { _,
51 isChecked ->
52             roundUp = isChecked
53             updateTip()
54         }
55
56         updateTip()
57     }
58
59     private fun updateTip() {
60         val amountDouble = amount.toDoubleOrNull() ?: 0.0
61         val tipPercent = when (tipOption) {
62             "15%" -> 0.15
63             "18%" -> 0.18
64             "20%" -> 0.20
65             else -> 0.15
66         }
67     }

```

59	
60	var tip = amountDouble * tipPercent
61	if (roundUp) tip = ceil(tip)
62	
63	val formattedTip =
	NumberFormat.getCurrencyInstance().format(tip)
64	
65	if (formattedTip == "\$0.00") {
66	binding.tipAmount.text = "Tip Amount: \$0.00"
67	binding.tipAmount.textAlignment =
	View.TEXT_ALIGNMENT_CENTER
68	} else {
69	binding.tipAmount.text = "Tip
	Amount:\n\$formattedTip"
70	binding.tipAmount.textAlignment =
	View.TEXT_ALIGNMENT_VIEW_START
71	}
72	}
73	}

Tabel 1. Source Code Jawaban Soal 1 MainActivity.kt XML

activity_main.xml

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.constraintlayout.widget.ConstraintLayout
	xmlns:android="http://schemas.android.com/apk/res/android"
3	xmlns:app="http://schemas.android.com/apk/res-auto"
4	xmlns:tools="http://schemas.android.com/tools"
5	android:id="@+id/main"
6	android:layout_width="match_parent"
7	android:layout_height="match_parent"
8	tools:context=".MainActivity">
9	
10	<TextView
11	android:id="@+id/textView"
12	android:layout_width="wrap_content"
13	android:layout_height="wrap_content"
14	android:layout_marginStart="48dp"
15	android:layout_marginTop="148dp"
16	android:text="Calculate Tip"
17	android:textSize="16sp"
18	app:layout_constraintStart_toStartOf="parent"
19	app:layout_constraintTop_toTopOf="parent" />
20	
21	<com.google.android.material.textfield.TextInputLayout
	android:id="@+id/billAmount"
22	
23	style="@style/Widget.MaterialComponents.TextInputLayout.FilledBox"
24	android:layout_width="match_parent"
25	android:layout_height="wrap_content"
26	android:layout_marginStart="46dp"

```

27         android:layout_marginTop="12dp"
28         android:layout_marginEnd="46dp"
29         android:hint="@string/bill_amount"
30         app:startIconDrawable="@drawable/money"
31         app:layout_constraintTop_toBottomOf="@id/textView"
32         app:layout_constraintStart_toStartOf="parent"
33         app:layout_constraintEnd_toEndOf="parent">
34
35     <com.google.android.material.textfield.TextInputEditText
36         android:id="@+id/textInputEditText"
37         android:layout_width="match_parent"
38         android:layout_height="wrap_content"
39         android:inputType="numberDecimal" />
40     </com.google.android.material.textfield.TextInputLayout>
41
42     <com.google.android.material.textfield.TextInputLayout
43         android:id="@+id/tipPercentage"
44         style="@style/Widget.MaterialComponents.TextInputLayout.FilledBox.ExposedDropdownMenu"
45         android:layout_width="match_parent"
46         android:layout_height="wrap_content"
47         android:layout_marginStart="46dp"
48         android:layout_marginTop="16dp"
49         android:layout_marginEnd="46dp"
50         android:hint="@string/how_was_the_service"
51         app:startIconDrawable="@drawable/percent"
52         app:layout_constraintTop_toBottomOf="@id/billAmount"
53         app:layout_constraintStart_toStartOf="parent"
54         app:layout_constraintEnd_toEndOf="parent">
55
56         <AutoCompleteTextView
57             android:id="@+id/autoCompleteTextView"
58             android:layout_width="match_parent"
59             android:layout_height="wrap_content"
60             android:inputType="none"/>
61     </com.google.android.material.textfield.TextInputLayout>
62
63     <TextView
64         android:id="@+id/textView3"
65         android:layout_width="wrap_content"
66         android:layout_height="wrap_content"
67         android:layout_marginStart="44dp"
68         android:layout_marginTop="34dp"
69         android:layout_marginBottom="453dp"
70         android:text="@string/round_up_tip"
71         app:layout_constraintBottom_toBottomOf="parent"
72         app:layout_constraintStart_toStartOf="parent"
73         app:layout_constraintTop_toBottomOf="@+id/tipPercentage"
74     />
75
76     <androidx.appcompat.widget.SwitchCompat

```

76	android:id="@+id/switch1"
77	android:layout_width="wrap_content"
78	android:layout_height="wrap_content"
79	android:layout_marginTop="20dp"
80	android:layout_marginEnd="46dp"
81	android:layout_marginBottom="438dp"
82	android:thumb="@drawable/thumb"
83	app:track="@drawable/track"
84	app:layout_constraintBottom_toBottomOf="parent"
85	app:layout_constraintEnd_toEndOf="parent"
86	app:layout_constraintTop_toBottomOf="@+id/tipPercentage"
	/>
87	
88	<TextView
89	android:id="@+id/tipAmount"
90	android:layout_width="0dp"
91	android:layout_height="wrap_content"
92	android:layout_marginStart="48dp"
93	android:layout_marginTop="84dp"
94	android:layout_marginEnd="80dp"
95	android:layout_marginBottom="292dp"
96	android:gravity="start"
97	android:text="Tip Amount: \$0.00"
98	android:textAlignment="viewStart"
99	android:textSize="34sp"
100	android:textStyle="bold"
101	app:layout_constraintBottom_toBottomOf="parent"
102	app:layout_constraintEnd_toEndOf="parent"
103	app:layout_constraintStart_toStartOf="parent"
104	app:layout_constraintTop_toBottomOf="@+id/tipPercentage" />
105	</androidx.constraintlayout.widget.ConstraintLayout>

Tabel 2. Source Code Jawaban Soal 1 activity_main XML

Compose:

MainActivity.kt

1	package com.example.tiptime
2	
3	import android.os.Bundle
4	import androidx.activity.ComponentActivity
5	import androidx.activity.compose.setContent
6	import androidx.activity.enableEdgeToEdge
7	import androidx.annotation.DrawableRes
8	import androidx.compose.foundation.layout.Arrangement
9	import androidx.compose.foundation.layout.Column
10	import androidx.compose.foundation.layout.Spacer
11	import androidx.compose.foundation.layout.fillMaxSize
12	import androidx.compose.foundation.layout.fillMaxWidth
13	import androidx.compose.foundation.layout.height
14	import androidx.compose.foundation.layout.padding

```

15 import androidx.compose.foundation.layout.safeDrawingPadding
16 import androidx.compose.foundation.layout.statusBarsPadding
17 import androidx.compose.foundation.rememberScrollState
18 import androidx.compose.foundation.text.KeyboardOptions
19 import androidx.compose.foundation.verticalScroll
20 import androidx.compose.material3.MaterialTheme
21 import androidx.compose.material3.Surface
22 import androidx.compose.material3.Text
23 import androidx.compose.material3.TextField
24 import androidx.compose.runtime.Composable
25 import androidx.compose.runtime.mutableStateOf
26 import androidx.compose.runtime.remember
27 import androidx.compose.runtime.setValue
28 import androidx.compose.runtime.getValue
29 import androidx.compose.ui.Alignment
30 import androidx.compose.ui.Modifier
31 import androidx.compose.ui.res.stringResource
32 import androidx.compose.ui.text.input.KeyboardType
33 import androidx.compose.ui.tooling.preview.Preview
34 import androidx.compose.ui.unit.dp
35 import com.example.tiptime.ui.theme.TipTimeTheme
36 import java.text.NumberFormat
37 import androidx.annotation.StringRes
38 import androidx.compose.foundation.layout.Row
39 import androidx.compose.ui.text.input.ImeAction
40 import androidx.compose.material3.ExposedDropDownMenuBox
41 import
    androidx.compose.material3.ExposedDropDownMenuDefaults
42 import androidx.compose.material3.Switch
43 import androidx.compose.material3.Icon
44 import androidx.compose.ui.res.painterResource
45 import androidx.compose.material3.ExperimentalMaterial3Api
46
47
48 class MainActivity : ComponentActivity() {
49     override fun onCreate(savedInstanceState: Bundle?) {
50         enableEdgeToEdge()
51         super.onCreate(savedInstanceState)
52         setContent {
53             TipTimeTheme {
54                 Surface(
55                     modifier = Modifier.fillMaxSize(),
56                 ) {
57                     TipTimeLayout()
58                 }
59             }
60         }
61     }
62 }
63
64 @OptIn(ExperimentalMaterial3Api::class)
65 @Composable

```

```

66 fun TipTimeLayout() {
67     var amountInput by remember { mutableStateOf("") }
68     val amount = amountInput.toDoubleOrNull() ?: 0.0
69
70     var dropDownTipPercent by remember { mutableStateOf(15.0)
71 }
72
73     var roundUp by remember { mutableStateOf(false) }
74     val tip = calculateTip(amount, dropDownTipPercent,
75 roundUp)
76     Column(
77         modifier = Modifier
78             .statusBarsPadding()
79             .padding(horizontal = 40.dp)
80             .verticalScroll(rememberScrollState())
81             .safeDrawingPadding(),
82         horizontalAlignment = Alignment.CenterHorizontally,
83         verticalArrangement = Arrangement.Center
84     ) {
85         Text(
86             text = stringResource(R.string.calculate_tip),
87             modifier = Modifier
88                 .padding(bottom = 16.dp, top = 40.dp)
89                 .align(alignment = Alignment.Start)
90         )
91         EditNumberField(
92             label = R.string.bill_amount,
93             leadingIcon = R.drawable.money,
94             keyboardOptions = KeyboardOptions.Default.copy(
95                 keyboardType = KeyboardType.Number,
96                 imeAction = ImeAction.Next
97             ),
98             value = amountInput,
99             onValueChange = { amountInput = it },
100             modifier = Modifier
101                 .padding(bottom = 32.dp)
102                 .fillMaxWidth()
103         )
104         TipPercentageDropdown(
105             selectedTip = dropDownTipPercent,
106             leadingIcon = R.drawable.percent,
107             onTipSelected = { dropDownTipPercent = it },
108             modifier = Modifier
109                 .padding(bottom = 32.dp)
110                 .fillMaxWidth()
111         )
112         RoundTheTipRow(
113             roundUp = roundUp,
114             onRoundUpChanged = { roundUp = it },
115             modifier = Modifier.padding(bottom = 32.dp)
116         )
117     }
118 }

```

```

115         val numericTip = tip.replace(Regex("[^\\d.]"),
116         "").toDoubleOrNull() ?: 0.0
117         if (numericTip == 0.0) {
118             Text(
119                 text = stringResource(R.string.tip_amount,
120                 tip),
121                 style
122                 =
123                 MaterialTheme.typography.displaySmall
124             )
125         }
126         else {
127             Column (modifier
128             =
129             Modifier.align(Alignment.Start)) {
130                 Text(
131                     text = "Tip Amount: ",
132                     style
133                     =
134                     MaterialTheme.typography.displaySmall
135                 )
136                 Text(
137                     text = tip,
138                     style
139                     =
140                     MaterialTheme.typography.displaySmall
141                 )
142             }
143         }
144         Spacer(modifier = Modifier.height(150.dp))
145     }
146 }
147
148 @Composable
149 fun EditNumberField (
150     @StringRes label: Int,
151     @DrawableRes leadingIcon: Int,
152     keyboardOptions: KeyboardOptions,
153     value: String,
154     onValueChange: (String) -> Unit,
155     modifier: Modifier = Modifier
156 ) {
157     TextField(
158         label = { Text(stringResource(label)) },
159         singleLine = true,
160         keyboardOptions = keyboardOptions,
161         value = value,
162         leadingIcon = { Icon(painter = painterResource(id =
163         leadingIcon), null) },
164         onValueChange = onValueChange,
165         modifier = modifier
166     )
167 }
168
169 @OptIn(ExperimentalMaterial3Api::class)
170 @Composable

```

```

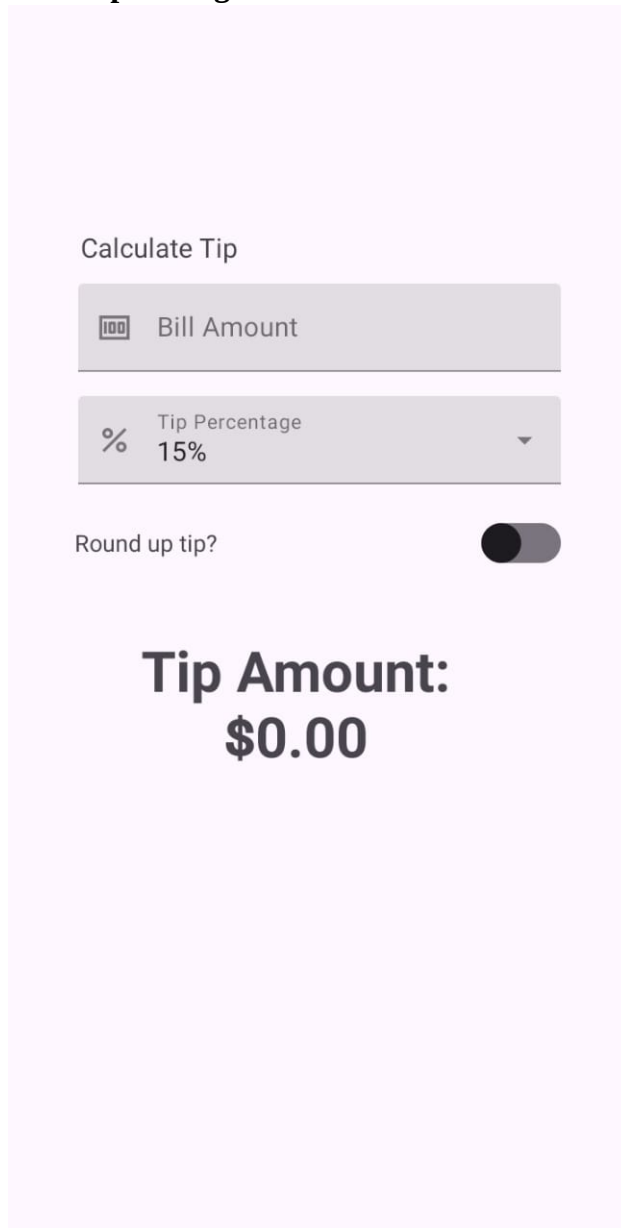
160 fun TipPercentageDropdown(
161     @DrawableRes leadingIcon: Int,
162     selectedTip: Double,
163     onTipSelected: (Double) -> Unit,
164     modifier: Modifier = Modifier
165 ) {
166     val tipOptions = listOf(15.0, 18.0, 20.0)
167     var expanded by remember { mutableStateOf(false) }
168     val selectedText = "${selectedTip.toInt()}%"
169
170
171     ExposedDropdownMenuBox(
172         expanded = expanded,
173         onExpandedChange = { expanded = !expanded },
174         modifier = modifier
175     ) {
176         TextField(
177             readOnly = true,
178             value = selectedText,
179             leadingIcon = { Icon(painter = painterResource(id
180 = leadingIcon), null) },
181             onValueChange = {},
182             label = Text(stringResource(R.string.how_was_the_service)) },
183             trailingIcon = {
184                 ExposedDropdownMenuDefaults.TrailingIcon(
185                     expanded = expanded
186                 )
187             },
188             modifier = Modifier
189                 .menuAnchor()
190                 .fillMaxWidth()
191         )
192         ExposedDropdownMenu(
193             expanded = expanded,
194             onDismissRequest = { expanded = false }
195         ) {
196             tipOptions.forEach { tip ->
197                 androidx.compose.material3.DropdownMenuItem(
198                     text = { Text("${tip.toInt()}%") },
199                     onClick = {
200                         onTipSelected(tip)
201                         expanded = false
202                     }
203                 )
204             }
205         }
206     }
207 }
208 @Composable
209 fun RoundTheTipRow(
    roundUp: Boolean,

```


210	onRoundUpChanged: (Boolean) -> Unit,
211	modifier: Modifier = Modifier) {
212	Row(
213	modifier = modifier
214	.fillMaxWidth()
215	.padding(vertical = 8.dp),
216	verticalAlignment = Alignment.CenterVertically,
217	horizontalArrangement = Arrangement.SpaceBetween
218	) {
219	Text(text = stringResource(R.string.round_up_tip))
220	Switch(
221	checked = roundUp,
222	onCheckedChange = onRoundUpChanged
223	)
224	}
225	}
226	
227	
228	
229	private fun calculateTip(
230	amount: Double,
231	tipPercent: Double,
232	roundUp: Boolean): String {
233	var tip = tipPercent / 100 * amount
234	if(roundUp) {
235	tip = kotlin.math.ceil(tip)
236	}
237	return NumberFormat.getCurrencyInstance().format(tip)
238	}
239	
240	@Preview(showBackground = true)
241	@Composable
242	fun TipTimeLayoutPreview() {
243	TipTimeTheme {
244	TipTimeLayout()
245	}
246	}

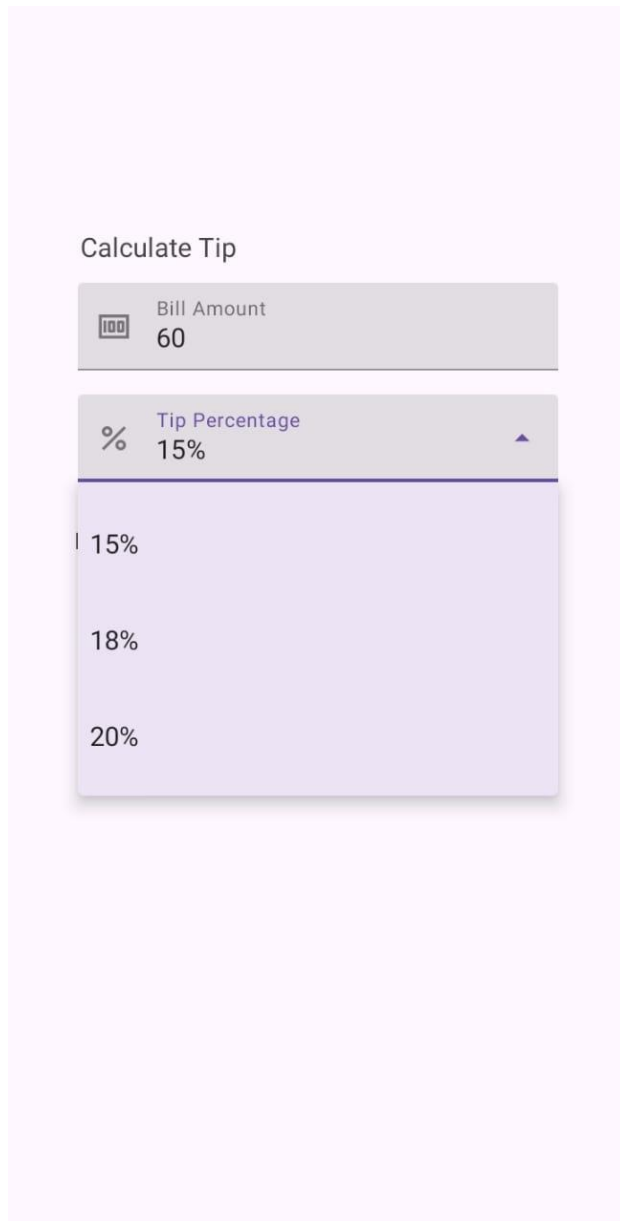
Tabel 3. Source Code Jawaban Soal 1 MainActivity.kt Compose

B. Output Program



The screenshot shows a mobile application interface for calculating a tip. At the top, the title 'Calculate Tip' is displayed. Below it, there are two input fields: 'Bill Amount' with a currency icon (100) and 'Tip Percentage' with a percentage icon (%) and a dropdown arrow. The 'Tip Percentage' field is currently set to '15%'. Below these fields, there is a toggle switch labeled 'Round up tip?'. The toggle switch is currently turned off. At the bottom, the result is displayed in large, bold text: 'Tip Amount: \$0.00'.

Gambar 4. Screenshot Hasil Jawaban Soal 1 Tampilan Awal Aplikasi XML



Gambar 5. Screenshot Hasil Jawaban Soal 1 Tampilan Pilihan Persentase Tip XML

Calculate Tip

 Bill Amount

60

 Tip Percentage

18%



Round up tip? ☒

Tip Amount:
\$11.00

Gambar 6. Screenshot Hasil Jawaban Soal 1 Tampilan Aplikasi Setelah Dijalankan XML

Calculate Tip



Bill Amount



Tip Percentage

15%



Round up tip?



Tip Amount: \$0.00

Gambar 7. Screenshot Hasil Jawaban Soal 1 Tampilan Awal Aplikasi Compose

Calculate Tip

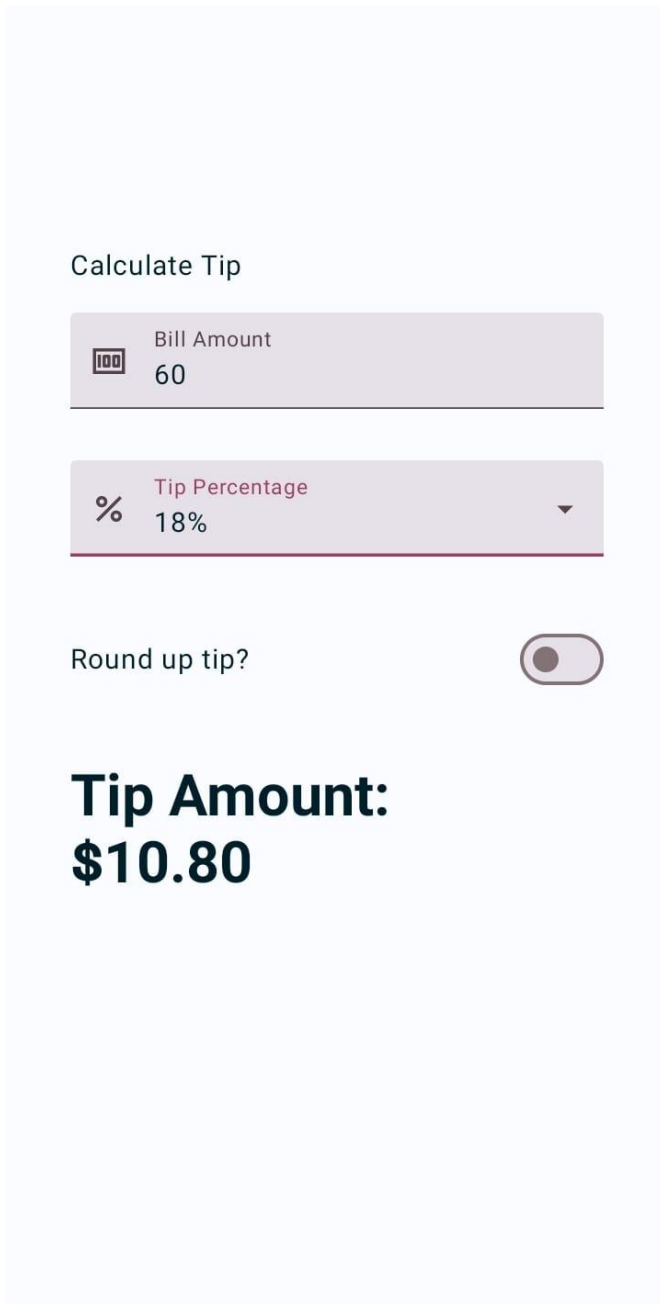
 Bill Amount
60

% Tip Percentage 15% ▲

- 15%
- 18%
- 20%

\$9.00

Gambar 8. Screenshot Hasil Jawaban Soal 1 Tampilan Pilihan Persentase Tip Compose



Gambar 9. Screenshot Hasil Jawaban Soal 1 Tampilan Aplikasi Setelah Dijalankan Compose

C. Pembahasan

XML:

MainActivity.kt:

Pada line 1, dideklarasikan nama package file Kotlin

Pada line 3, diimport bundle untuk menyimpan data sementara antar lifecycle seperti onCreate, onSaveInstanceState, dll.

Pada line 4, diimportt Editable untuk mendeteksi perubahan teks pada input (EditText).

Pada line 5, diimportt View untuk mengatur property tampilan seperti textAlignment.

Pada line 6, diimportt AppCompatActivity yang merupakan superclass dari activity yang menyediakan fitur-fitur android.

Pada line 7, diimportt ActivityMainBinding, kelas binding otomatis yang dihasilkan dari layout XML oleh fitur View Binding.

Pada line 8, diimportt NumberFormat, untuk memformat angka jadi format mata uang.

Pada line 10, diimportt math.ceil untuk membulatkan angka ke atas.

Pada line 12, MainActivity adalah activity utama (layar utama) yang merupakan turunan dari AppCompatActivity sehingga merwarisi fitur-fitur activity android.

Pada line 14, variabel binding menyambungkan elemen-elemen dari XML ke Kotlin menggunakan View Binding, variabel ini tidak langsung diinisialisasi dan hanya bisa diakses dalam class MainActivity.

Pada line 15, deklarasi variabel amount menyimpan input jumlah tagihan dari pengguna sebagai String, variabel ini hanya bisa diakses dalam class MainActivity.

Pada line 16, deklarasi variabel tipOption menyimpan persentase tip yang dipilih dari dropdown (default 15%), variabel ini hanya bisa diakses dalam class MainActivity.

Pada line 17, deklarasi variabel roundup Boolean yang menunjukkan apakah tip harus dibulatkan ke atas, variabel ini hanya bisa diakses dalam class MainActivity.

Pada line 19-22, function onCreate merupakan method utama yang dipanggil saat activity pertama kali dibuat. Binding untuk menghubungkan layout dengan file Kotlin. setContentView(binding.root) menentukan layout utama dari activity ini.

Pada line 24-27, inisialisasi dropdown persentase tip. Variabel tipOptions berisi pilihan persentase tip menggunakan function listOf(). ArrayAdapter digunakan untuk menghubungkan data list ke AutoCompleteTextView. setAdapter() menghubungkan dropdown dengan list. setText(tipOption, false) untuk menampilkan pilihan default yaitu 15%.

Pada line 29-36, terdapat listener perubahan input tagihan. Menggunakan TextWatcher untuk mendeteksi perubahan input. Setelah teks berubah (afterTextChanged), nilai amount diperbarui dan updateTip() dipanggil untuk menghitung ulang tip.

Pada line 38-41, terdapat listener pemilihan dropdown, saat user memilih persentase dari dropdown, tipOption diupdate dan tip dihitung ulang.

Pada line 43-48, terdapat listener switch pembulatan tip, mengatur apakah tip akan dibulatkan ke atas atau tidak. Lalu, langsung memanggil updateTip() setelah status switch berubah..

Pada line 51-70, terdapat function update() sebagai logika inti untuk update tagihan, variable; amountDouble mengonversi input tagihan dari String ke Double. Jika input kosong atau bukan angka, maka return 0.0. lalu variabel tipPercent menentukan nilai persentase tip sesuai pilihan user. Variabel tip menghitung tip berdasarkan jumlah tagihan dan persentase. Jika roundup aktif, gunakan ceil() untuk membulatkan angka ke atas. Lalu format nilai tip ke dalam bentuk mata uang, jika hasil tip ada %0.00, posisikan teks di Tengah, jika selain itu maka tampilkan rata kiri.

activity_main.xml:

Pada line 1, terdapat versi XML dan encoding yang digunakan

Pada line 2, dideklarasikan root dari file layout, yakni ConstraintLayout dan namespace XML untuk atribut Android

Pada line 3, dideklarasikan namespace untuk atribut kustom

Pada line 4, dideklarasikan namespace untuk atribut tools dalam aplikasi

Pada line 5-8, android:id: ID layout (jarang dipakai langsung, tapi bisa untuk referensi jika dibutuhkan). layout_width dan layout_height: match_parent berarti layout mengisi seluruh layar. tools:context: memberitahu Android Studio bahwa layout ini milik MainActivity (digunakan untuk preview).

Pada line 10-19, terdapat element <TextView> adalah label yang menampilkan teks "Calculate Tip". Atribut android:id="@+id/textView" memberikan ID agar bisa diakses dari kode Kotlin. layout_width dan layout_height di-set ke wrap_content, artinya ukuran elemen akan mengikuti kontennya (teks). Margin kiri layout_marginStart="48dp" dan margin atas layout_marginTop="148dp" memberikan jarak terhadap tepi parent layout. Atribut android:text="Calculate Tip" adalah teks yang ditampilkan. textSize="16sp" mengatur ukuran teks dalam skala yang mengikuti preferensi ukuran font pengguna. Dua atribut terakhir adalah constraint: app:layout_constraintStart_toStartOf="parent" membuat sisi kiri TextView sejajar dengan sisi kiri ConstraintLayout, dan

`app:layout_constraintTop_toTopOf="parent"` berarti bagian atas `TextView` sejajar dengan atas layout.

Pada line 21-40, terdapat element `<TextInputLayout>` untuk input teks dengan label dan ikon. `android:id="@+id/billAmount"` adalah ID dari element ini. `style="@style/Widget.MaterialComponents.TextInputLayout.FilledBox"` artinya menggunakan style `filledBox`. `android:layout_width="match_parent"` artinya selebar layar. `android:layout_height="wrap_content"` artinya tinggi mengikuti isinya. `android:layout_marginStart="46dp"` maka jarak dari sisi kiri 46dp. `android:layout_marginTop="12dp"` artinya jarak atas dari elemen sebelumnya adalah 12dp. `android:layout_marginEnd="46dp"` artinya jarak dari sisi kanan adalah 46dp. `android:hint="@string/bill_amount"` maka hint akan muncul di dalam input field (kotak input). `app:startIconDrawable="@drawable/money"` untuk menampilkan ikon uang di sebelah kiri input. `app:layout_constraintTop_toBottomOf="@id/textView"` maka element ditaruh di bawah elemen dengan ID `textView`. `app:layout_constraintStart_toStartOf="parent"` untuk sejajar kiri parent. `app:layout_constraintEnd_toEndOf="parent"` untuk sejajar kanan parent. Di dalamnya terdapat `<TextInputEditText />` yang dimana tempat user mengetik jumlah uang. `android:id="@+id/textInputEditText"` adalah ID untuk diakses dari Kotlin. `android:layout_width="match_parent"` maka selebar kotak input. `android:layout_height="wrap_content"` maka tinggi otomatis sesuai dengan isi konten. `android:inputType="numberDecimal"` maka hanya menerima angka desimal.

Pada line 42-61, terdapat element `<TextInputLayout>` kedua, untuk memilih persentase tip. ID-nya `@+id/tipPercentage`, menggunakan style `ExposedDropdownMenu` agar input bisa menampilkan dropdown. Lebar `match_parent`, tinggi `wrap_content`, margin kiri dan kanan sama yaitu 46dp, margin atas dari elemen sebelumnya adalah 16dp. Hint-nya adalah teks dari string resource `@string/how_was_the_service`. Ikon `@drawable/percent` ditampilkan di kiri input. Elemen ini di-constraint ke bawah `billAmount`, dan kiri-kanannya sejajar parent. Di dalamnya ada `AutoCompleteTextView` dengan ID `autoCompleteTextView`, tipe input-nya diset ke `none` karena pengguna memilih dari dropdown, bukan mengetik langsung.

Pada line 63-73, terdapat element `<TextView>` untuk label switch "Round up tip". ID-nya `textView3`, ukuran mengikuti konten. Margin kiri 44dp, margin atas 34dp, dan margin bawah

453dp (cukup besar agar tidak mepet ke bawah layar). Teks diambil dari string resource @string/round_up_tip. Elemen ini di-constraint ke bawah tipPercentage, sisi kiri sejajar parent, dan bagian bawah diset ke bawah parent (supaya tetap berada di area bawah layar). Pada line 75-86, terdapat element <SwitchCompat> yaitu switch modern dari AndroidX. ID-nya @+id/switch1. Ukuran mengikuti konten. Margin atas 20dp, margin kanan 46dp, margin bawah 438dp. Atribut android:thumb="@drawable/thumb" dan app:track="@drawable/track" mengatur gambar thumb (lingkaran pada switch) dan track (jalur switch) menggunakan drawable custom. Constraint top-nya di bawah tipPercentage, kanan sejajar parent, bawah ke bawah parent.

Pada line 88-105, terdapat element <TextView> untuk menampilkan hasil perhitungan tip. ID-nya tipAmount, lebar di-set 0dp (dalam ConstraintLayout, ini artinya lebar akan mengikuti constraint antara kiri dan kanan), tinggi wrap_content. Margin kiri 48dp, atas 84dp, kanan 80dp, bawah 292dp. Atribut gravity="start" mengatur posisi teks agar sejajar ke kiri. text="Tip Amount: \$0.00" adalah nilai default. textAlignment="viewStart" untuk memastikan alignment teks mengikuti arah tampilan. Ukuran teks besar (34sp) dan textStyle="bold" untuk menonjolkan hasil. Constraint-nya menyambung dari bawah tipPercentage, lalu kiri-kanan dan bawah sejajar dengan parent.

Compose:

MainActivity.kt:

Pada line 1, dideklarasikan nama package file Kotlin.

Pada line 3, diimport Bundle untuk mengkomunikasikan data antar aktivitas.

Pada line 4, diimport ComponentActivity, superclass untuk aktivitas di Jetpack Compose yang menggantikan AppCompatActivity.

Pada line 5, diimport setContent untuk menetapkan konten berbasis Compose dalam activity.

Pada line 6, diimport enableEdgeToEdge untuk memungkinkan tampilan UI mengisi seluruh layar, termasuk area status bar.

Pada line 7, diimport DrawableRes untuk menandai parameter sebagai resource gambar drawable.

Pada line 8-16, diimport berbagai komponen layout (Arrangement, Column, Spacer, dll.)

untuk mengatur tampilan UI dalam Compose. Pada line 17, diimport rememberScrollState untuk membuat dan mengingat status scroll dalam UI.

Pada line 18, diimport KeyboardOptions untuk mengonfigurasi opsi keyboard pada input field.

Pada line 19, diimport verticalScroll untuk memungkinkan elemen UI di-scroll secara vertikal.

Pada line 20-23, diimport berbagai komponen Material 3 (MaterialTheme, Surface, Text, TextField, dll.) untuk membangun UI berbasis Material Design 3.

Pada line 24-28, diimport komponen reaktivitas Compose seperti mutableStateOf, remember, dan setValue untuk menangani state dalam UI.

Pada line 29-35, diimport komponen UI (Alignment, Modifier, dll.) untuk penataan dan pengaturan posisi elemen UI.

Pada line 32, diimport KeyboardType untuk menentukan jenis keyboard yang digunakan dalam input field.

Pada line 31, diimport stringResource untuk mengambil resource string dalam UI.

Pada line 33, diimport Preview untuk melihat preview komponen Compose dalam editor.

Pada line 34, diimport dp untuk mendefinisikan ukuran dalam satuan density-independent pixels.

Pada line 35, diimport tema TipTimeTheme untuk menentukan tema aplikasi.

Pada line 36, diimport NumberFormat untuk memformat angka menjadi format mata uang.

Pada line 37, diimport StringRes untuk menandai parameter sebagai resource string yang diambil dari file strings.xml.

Pada line 38, diimport Row untuk menata elemen secara horizontal dalam layout Compose.

Pada line 39, diimport ImeAction untuk menentukan aksi keyboard seperti "Next" atau "Done".

Pada line 40, diimport ExposedDropdownMenuBox untuk menampilkan dropdown menu yang dapat diekspos.

Pada line 41, diimport ExposedDropdownMenuDefaults untuk mengatur pengaturan default pada ExposedDropdownMenu.

Pada line 42, diimport Switch untuk menampilkan komponen switch (aktif/tidak aktif) dalam

UI.

Pada line 43, diimport Icon untuk menampilkan ikon dalam komponen UI. Pada line 44, diimport painterResource untuk mengambil resource gambar sebagai objek Painter.

Pada line 45, diimport ExperimentalMaterial3Api untuk menandai penggunaan API eksperimental dari Material 3.

Pada line 48-62, kelas MainActivity meng-extend ComponentActivity, yang merupakan kelas dasar untuk activity di Jetpack Compose. Di dalam method onCreate, fungsi enableEdgeToEdge() digunakan untuk mengaktifkan tampilan layar penuh hingga ke area status bar. Setelah itu, super.onCreate(savedInstanceState) dipanggil untuk menjalankan logika bawaan dari ComponentActivity. Kemudian, setContent digunakan untuk menentukan tampilan Compose dengan TipTimeTheme, yang membungkus komponen UI dalam tema yang telah ditentukan sebelumnya. Di dalamnya, Surface digunakan sebagai pembungkus elemen UI untuk memberikan latar belakang dan memastikan seluruh layar terisi dengan Modifier.fillMaxSize(). Selanjutnya, fungsi TipTimeLayout() dipanggil untuk menampilkan layout utama aplikasi.

Pada line 64-136, TipTimeLayout adalah fungsi komponen utama dalam aplikasi ini yang menggunakan deklarasi UI berbasis Jetpack Compose. Fungsi ini dimulai dengan mendeklarasikan tiga state penting yang mengontrol input dan pilihan pengguna. Pertama, amountInput adalah variabel yang menyimpan input dari pengguna untuk jumlah tagihan, yang didefinisikan menggunakan remember dan mutableStateOf(""). remember memastikan bahwa nilai variabel ini akan dipertahankan saat tampilan diperbarui (misalnya, jika pengguna mengubah input). Nilai awalnya adalah string kosong, yang menunjukkan bahwa belum ada jumlah yang dimasukkan. amount adalah variabel turunan yang menghitung nilai numerik dari amountInput dengan mengonversi string ke tipe data Double menggunakan fungsi toDoubleOrNull(). Jika input tidak valid, nilai amount akan menjadi 0.0. Variabel dropDownTipPercent menyimpan persentase tip yang dipilih pengguna, dengan nilai default 15.0. roundUp adalah sebuah boolean yang menentukan apakah tip harus dibulatkan ke atas. Variabel ini dikelola menggunakan remember dan mutableStateOf(false) untuk menyimpan status pembulatan tip. Kemudian, fungsi calculateTip digunakan untuk menghitung nilai tip berdasarkan jumlah tagihan, persentase tip yang dipilih, dan apakah pembulatan harus

diterapkan. Di dalam UI, berbagai elemen ditampilkan menggunakan Column yang memiliki beberapa modifier untuk mengatur layout seperti `statusBarsPadding()` yang memberi ruang di bagian atas untuk status bar, `padding(horizontal = 40.dp)` untuk memberi jarak horizontal, dan `verticalScroll(rememberScrollState())` yang memungkinkan konten di dalam Column discroll secara vertikal jika diperlukan. Di dalam Column, terdapat elemen-elemen UI seperti Text untuk menampilkan judul, `EditNumberField` untuk input jumlah tagihan, `TipPercentageDropdown` untuk memilih persentase tip, dan `RoundTheTipRow` untuk memilih apakah tip harus dibulatkan. Hasil perhitungan tip kemudian ditampilkan dengan format mata uang menggunakan Text, dan jika tip yang dihitung adalah 0.0, hanya pesan "Tip Amount: 0" yang akan ditampilkan. Jika tip valid, maka dua baris teks ditampilkan, yaitu label "Tip Amount" dan nilai tip itu sendiri.

Pada line 138-156, `EditNumberField` adalah fungsi komponen yang mengatur tampilan input untuk jumlah tagihan. Fungsi ini menerima beberapa parameter: label, `leadingIcon`, `keyboardOptions`, `value`, `onValueChange`, dan modifier. Parameter label digunakan untuk menunjukkan teks yang menggambarkan input, yang akan diambil dari string resource. `leadingIcon` menunjukkan ikon yang muncul di sebelah kiri input, yang dalam hal ini adalah ikon uang yang diambil dari drawable resource. `keyboardOptions` memungkinkan penyesuaian keyboard yang digunakan saat pengguna berinteraksi dengan `TextField`. Di sini, keyboard dikonfigurasi untuk menerima input angka dengan tipe `KeyboardType.Number` dan aksi `ImeAction.Next`, yang memungkinkan pengguna untuk melanjutkan ke elemen input berikutnya. Parameter `value` berisi nilai input yang dimasukkan oleh pengguna, dan `onValueChange` adalah fungsi callback yang digunakan untuk memperbarui nilai `amountInput` saat pengguna mengubah teks. Modifier `padding` ditambahkan untuk memberi jarak vertikal dan horizontal di sekitar elemen input agar tampilan menjadi lebih rapi dan mudah dibaca. Modifier `fillMaxWidth()` memastikan bahwa `TextField` akan mengisi lebar layar secara penuh. Ini memungkinkan elemen input untuk responsif dan mudah diakses oleh pengguna.

Pada line 158-206, `TipPercentageDropdown` adalah fungsi komponen untuk menampilkan dropdown menu bagi pengguna untuk memilih persentase tip yang ingin dihitung. Fungsi ini menerima beberapa parameter, termasuk `leadingIcon`, `selectedTip`, `onTipSelected`, dan modifier. Parameter `leadingIcon` adalah ikon yang ditampilkan di samping dropdown, yang

berfungsi sebagai petunjuk visual bagi pengguna mengenai opsi yang dapat dipilih. `selectedTip` berisi nilai persentase tip yang saat ini dipilih, yang dalam hal ini didefinisikan dengan tipe `Double`. Fungsi callback `onTipSelected` digunakan untuk memperbarui nilai `dropDownTipPercent` ketika pengguna memilih persentase tip yang diinginkan. Di dalam fungsi ini, terdapat `ExposedDropdownMenuBox`, sebuah elemen UI yang memungkinkan pembuatan dropdown yang dapat dibuka dan ditutup. `ExposedDropdownMenuBox` mengontrol status terbuka atau tertutupnya dropdown dengan mengubah nilai `expanded`, yang dapat berubah ketika pengguna berinteraksi dengan dropdown. Di dalam dropdown menu, terdapat sejumlah opsi persentase tip (15%, 18%, 20%) yang ditampilkan menggunakan `DropdownMenuItem`. Ketika pengguna memilih salah satu item, nilai `dropDownTipPercent` diperbarui, dan dropdown ditutup. `TextField` di dalam `ExposedDropdownMenuBox` berfungsi hanya untuk menampilkan nilai persentase yang dipilih, dan pengaturan `readOnly = true` memastikan bahwa pengguna tidak dapat mengedit teks secara langsung. Ikon di bagian belakang dropdown ditambahkan menggunakan `ExposedDropdownMenuDefaults.TrailingIcon`, yang menunjukkan status dropdown apakah sedang terbuka atau tertutup.

Pada line 207-225, `RoundTheTipRow` adalah fungsi komponen yang menampilkan elemen `Row` untuk menampilkan pilihan apakah pengguna ingin membulatkan jumlah tip. Di dalam `Row`, terdapat elemen teks yang menunjukkan pilihan "Round Up Tip" diikuti dengan `Switch`. `Switch` adalah elemen UI yang memungkinkan pengguna untuk mengubah status dari `false` ke `true`, yang akan memperbarui nilai `roundUp`. Parameter `onCheckedChange` adalah callback yang digunakan untuk memperbarui status `roundUp` saat pengguna mengubah nilai `switch`. Modifier `padding` ditambahkan untuk memberi jarak vertikal di sekitar elemen `switch` agar tampil lebih rapi. Dengan menggunakan `Row`, elemen-elemen ini ditata secara horizontal dan diposisikan agar ada jarak antar elemen dengan `Arrangement.SpaceBetween`, yang memastikan bahwa teks dan `switch` memiliki ruang yang cukup di antara mereka.

Pada line 229-238, `calculateTip` adalah fungsi utilitas yang menghitung jumlah tip berdasarkan jumlah tagihan yang dimasukkan, persentase tip, dan status pembulatan. Fungsi ini menerima tiga parameter: `amount` (jumlah tagihan), `tipPercent` (persentase tip), dan `roundUp` (boolean yang menentukan apakah tip harus dibulatkan). Pertama, nilai tip dihitung dengan rumus $\text{tipPercent} / 100 * \text{amount}$. Jika `roundUp` bernilai `true`, maka nilai tip

dibulatkan ke atas menggunakan `kotlin.math.ceil()`. Fungsi `ceil` akan membulatkan angka tip ke angka bulat terdekat yang lebih besar atau sama dengan angka tersebut. Terakhir, hasil perhitungan tip diformat sebagai mata uang menggunakan `NumberFormat.getCurrencyInstance().format(tip)`. Fungsi ini mengembalikan string yang berisi nilai tip dalam format mata uang yang sesuai dengan lokal perangkat.

Pada line 240-246, `TipTimeLayoutPreview` adalah fungsi `@Preview` untuk melihat pratinjau dari tampilan UI di dalam IDE tanpa perlu menjalankan aplikasi pada perangkat. Dengan menambahkan anotasi `@Preview`, pengembang dapat melihat bagaimana tampilan `TipTimeLayout` akan muncul dengan tema `TipTimeTheme`. Ini membantu pengembang untuk memverifikasi dan menyempurnakan tampilan UI secara langsung dalam lingkungan pengembangan sebelum aplikasi dijalankan secara penuh.

D. Tautan Git

Berikut adalah tautan untuk source code yang telah dibuat.

[Praktikum_Mobile/Modul 2 at main · Bukhrii/Praktikum_Mobile](#)

SOAL 2

Soal Praktikum:

Jelaskan perbedaan dari implementasi XML dan Jetpack Compose beserta kelebihan dan kekurangan dari masing-masing implementasi.

A. Pembahasan

Dengan pendekatan ini Jetpack Compose, UI dibuat sebagai komposisi (compose) atau komponen dari elemen-elemen fungsional yang ditulis dalam Kotlin. Setiap komponen UI dihubungkan langsung dengan logika aplikasi menggunakan state-driven programming.

Sedangkan dengan pendekatan XML, UI dan logikanya dibuat secara terpisah, UI dibuat dalam file xml (main_activity.xml) dan untuk logikanya di file Kotlin. Untuk menghubungkannya dilakukan di file Kotlin menggunakan findViewById atau View Binding (direkomendasikan).

Beberapa contoh perbedaan implementasi:

1. Menampilkan teks "Calculate Tip":
 - Pada Jetpack Compose, menggunakan komponen Text yang disertai dengan modifier yang berupa objek Modifier untuk memodifikasi layout seperti mengatur padding.
 - Sedangkan pada XML, menggunakan element TextView, yang di mana posisi dan style ditentukan oleh atribut XML, misalnya atribut android:text dan android:layout_marginStart digunakan untuk menentukan teks dan margin. Namun, tidak ada keterikatan langsung antara UI dan state aplikasi dalam file XML.
2. Input (EditText) seperti Bill Amount:
 - Pada Jetpack Compose, untuk memasukkan jumlah tagihan menggunakan komponen TextField dalam bentuk fungsi compose EditNumberField(). Pengguna dapat langsung memasukkan nilai melalui UI ini. Fungsi mengelola input dengan menggunakan state (amountInput) yang di-update setiap kali pengguna memasukkan angka (nilai).

- Sedangkan Pada XML, menggunakan element `TextInputLayout` dan `TextInputEditText`, dengan pendekatan ini UI didefinisikan di file XML, namun perubahan input akan ditangani di kode Kotlin dengan `TextWatcher`

3. Dropdown untuk Persentase Tip:

- Pada Jetpack Compose, menggunakan `ExposedDropdownMenuBox`. UI diperbarui secara otomatis setiap kali pilihan persentase tip berubah.
- Sedangkan pada XML, menggunakan `AutoCompleteTextView` di dalam `TextInputLayout`. Interaksi ditangani di file Kotlin menggunakan `setOnClickListener` untuk memperbarui pilihan.

4. Switch untuk Pembulatan Tip:

- Pada Jetpack Compose, menggunakan komponen `Switch()`. Pilihan langsung terikat/terhubung langsung ke state `roundup` yang mengontrol apakah tip dibulatkan atau tidak
- Sedangkan pada XML, mendeklarasikan elemen `SwitchCompat` dalam XML. Interaksi pengguna dengan switch ditangani pengguna menggunakan `setCheckedChangeListener` di dalam kode Kotlin untuk mengubah status `roundup`.

Kelebihan dan Kekurangan Jetpack Compose

Kelebihan:

1. Bersifat deklaratif dan terintegrasi dengan logika aplikasi, sehingga lebih mudah untuk mengelola interaksi antara UI dan logika aplikasi.
2. Memungkinkan penulisan kode menjadi lebih simpel dan ringkas
3. UI lebih responsif dan dinamis, karena UI secara otomatis terupdate ketika state berubah
4. Pengelolaan layout yang lebih mudah dimodifikasi, karena dengan menggunakan fungsi-fungsi compose memungkinkan penggunaan kembali komponen UI secara lebih fleksibel.

Kekurangan:

1. Memerlukan sedikit waktu untuk memahami konsep-konsep baru seperti Modifier, state, dan remember, dll, bagi yang baru mempelajari pendekatan deklaratif.
2. Kurangnya kompatibilitas pendekatan sebelumnya sehingga membuat transisi penggunaan pendekatan menjadi lebih sulit
3. Ketergantungan pada versi Android dan Tools/Fitur karena terbilang masih baru dan belum sepenuhnya matang (masih perlu banyak pembaruan)

Kelebihan dan Kekurangan Jetpack Compose

Kelebihan:

1. Kompatibilitas lebih luas, karena banyak library UI dan komponen yang sudah tersedia sebelumnya dan stabil untuk digunakan
2. Lebih familiar bagi pengembang berpengalaman, karena termasuk pendekatan imperative (lebih duluan banyak digunakan sebelum declarative).
3. Tools dan Dokumentasi yang lengkap dan matang, karena sudah ada sejak lama.

Kekurangan:

1. Lebih sulit dikelola, karena UI dan logika aplikasi dibuat secara terpisah sehingga perlu mengelola secara lebih manual.
2. UI tidak terupdate secara otomatis (tidak responsive).
3. Kode lebih panjang dan rumit.